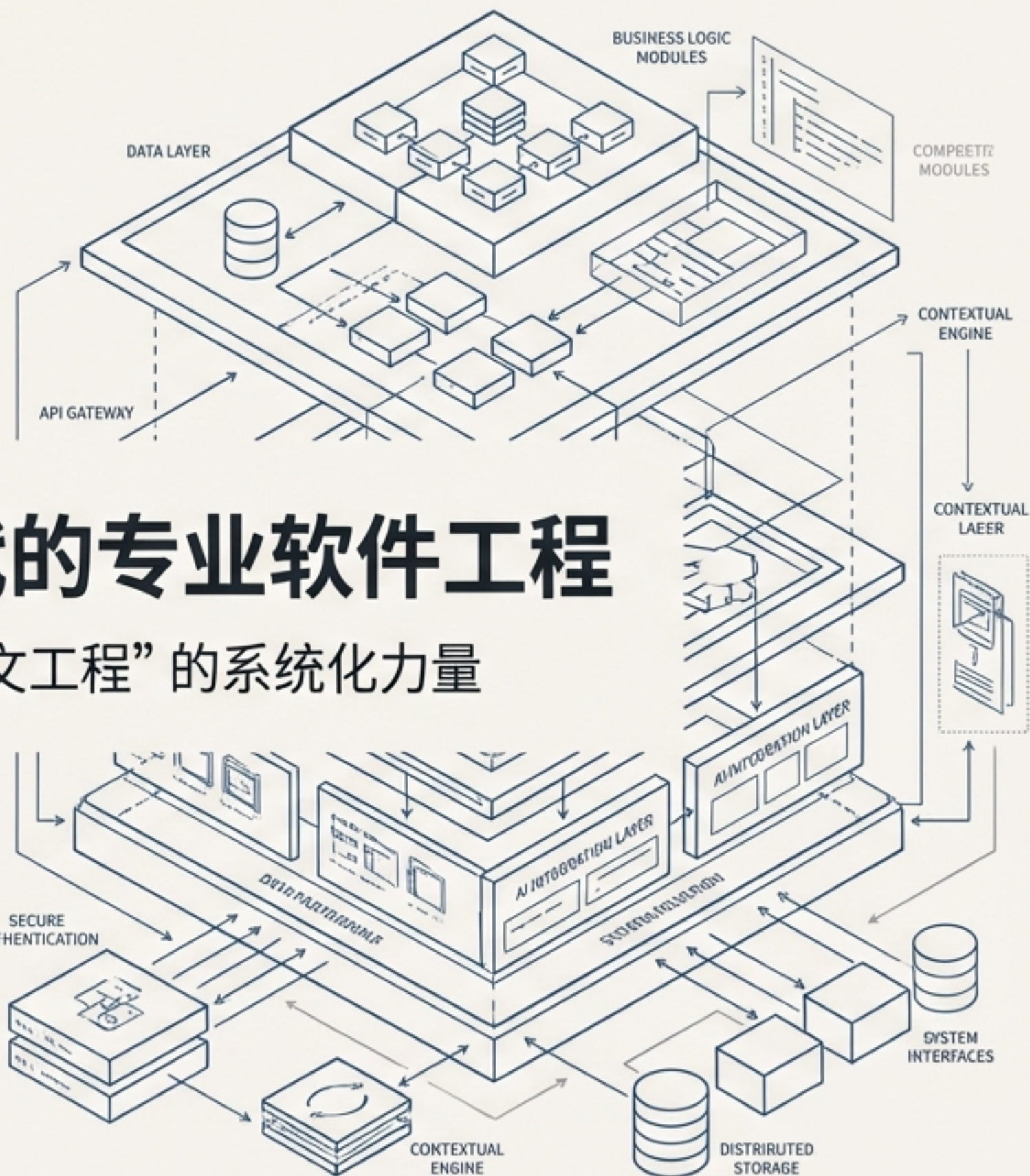


从直觉到架构：AI 时代的专业软件工程

超越“感觉式编程”，拥抱“上下文工程”的系统化力量



一切始于“感觉式编程” (Vibe Coding) 的蜜月期

定义

由 Andrej Karpathy 提出的术语，指严重依赖 AI 助手、凭直觉和反复试错来生成代码的工作方式。

体验

强调其带来的“即时代码生成的多巴胺冲击”。这是一种快速、有趣、充满探索性的体验。

适用场景

Martin Fowler 和 Kitze 指出，这种方法非常适合：

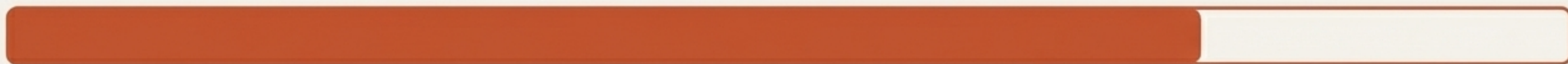
- 快速原型开发
- 周末的 Hackathon 项目
- 一次性脚本和探索性任务

“你不太关心代码细节，你按下接受，然后告诉 LLM 去做它需要做的事。”

— Andrej Karpathy (对 Vibe Coding 的描述)

但直觉无法规模化，当“感觉”撞上生产环境的现实

76.4%



根据 Codto 的《AI 代码质量现状》调查，76.4% 的开发者在没有人工审查的情况下，对上线 AI 生成的代码缺乏信心。

高幻觉率

AI 会自信地编造不存在的 API 或错误的逻辑。

低可维护性

生成的代码（“slop”）往往缺乏结构，难以调试和扩展。

学习循环中断

Martin Fowler 指出，不审查代码会跳过关键的学习过程，导致工程师无法真正理解和修改系统。

根本性的转变：我们正从确定性系统转向非确定性系统



这意味着我们不能再像对待传统编译器那样完全信任工具的输出。我们的工作重心必须从“编写代码”转向“验证、引导和构建系统来约束非确定性”。

“这是我职业生涯中见过的最大变革。其核心是从确定性到非确定性的转变。”
— Martin Fowler

解决方案：“上下文工程”（Context Engineering）的兴起

权威定义

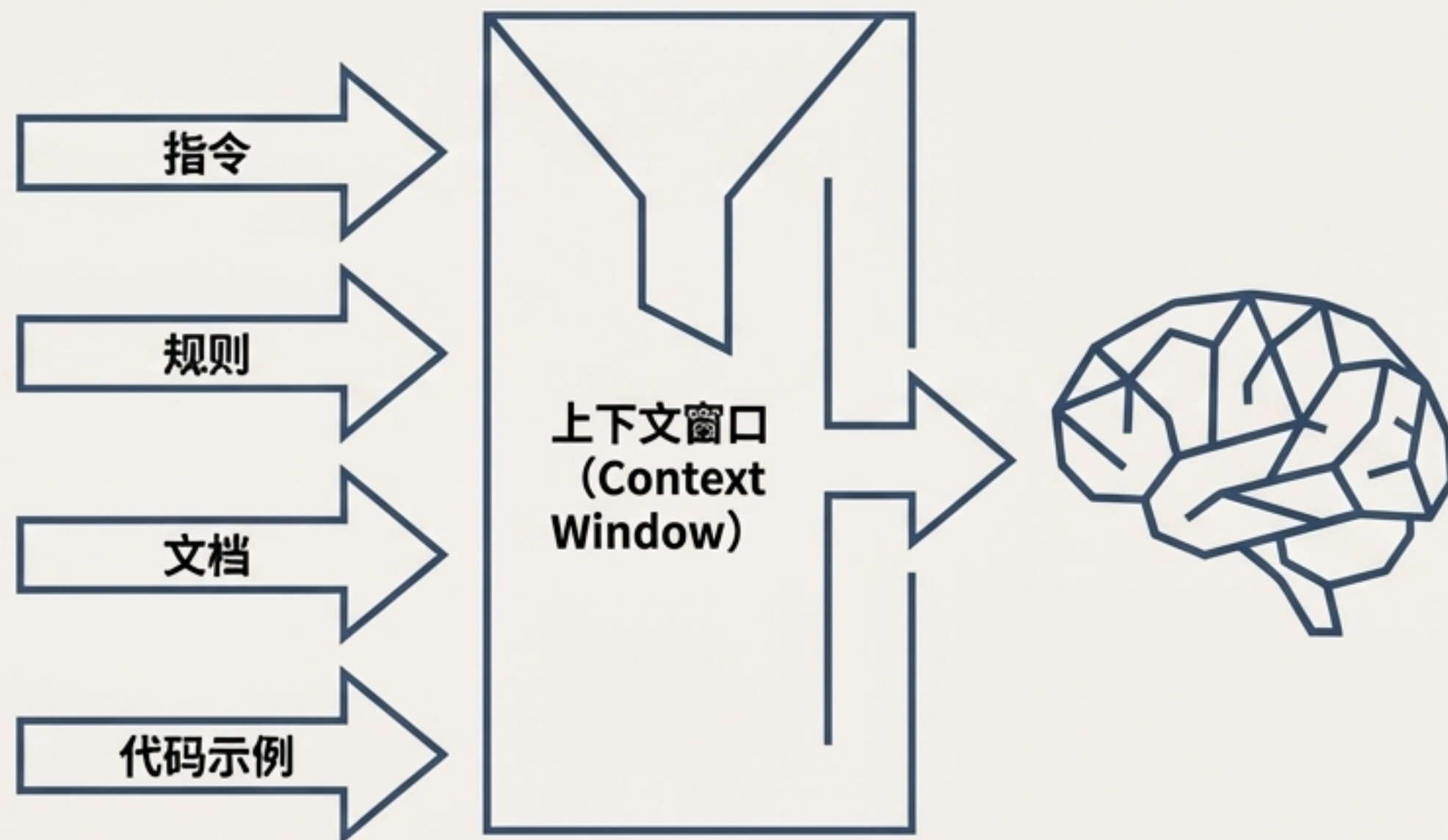
“提供所有上下文，使 LLM 有可能解决任务的艺术。” — Andrej Karpathy

上下文本身应被视为一种需要精心设计的工程资源。

与“感觉式编程”的对比

感觉式编程：依赖模型的“内部知识”和直觉。

上下文工程：将“外部知识”和结构化指令注入到工作流程中，由人类工程师设计和提供。



上下文工程远不止是“提示词工程”

提示词工程 (Prompt Engineering)



目标:

调整措辞和短语，以从 LLM 获得单个最佳回答。

范围:

战术性、局部的优化。

上下文工程 (Context Engineering)

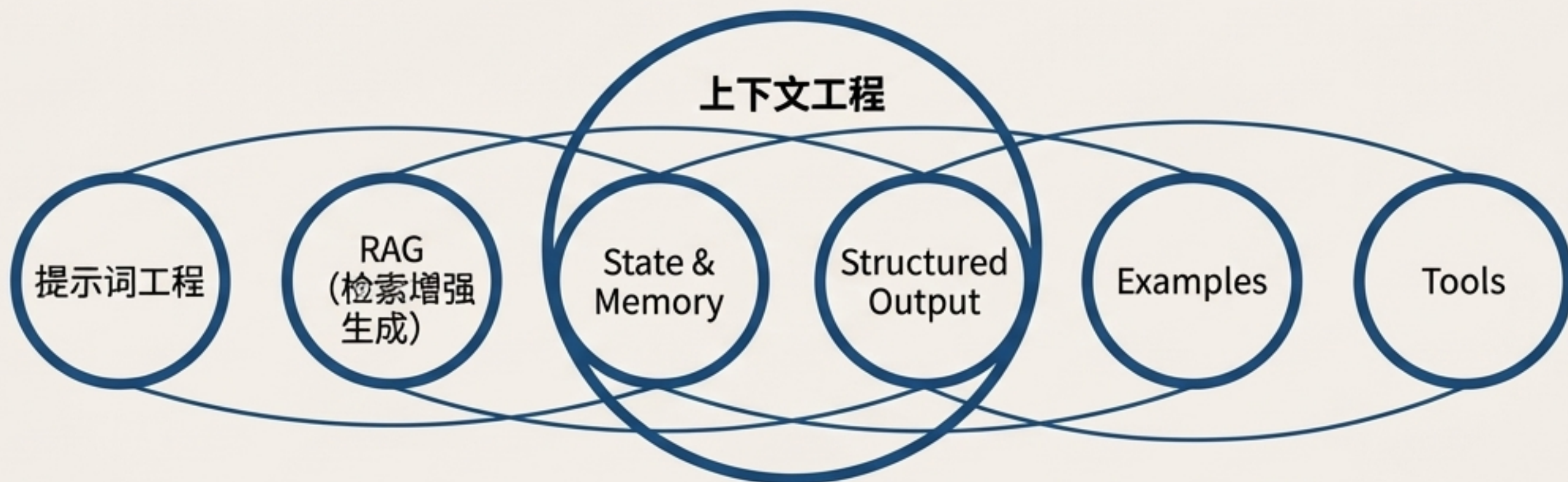


目标:

提供一个完整的、由事实、规则、文档、计划和工具组成的“上下文生态系统”。

范围:

战略性、系统化的构建。



“提示词工程是调整措辞。上下文工程是提供所有相关事实、规则、文档、计划、工具。”

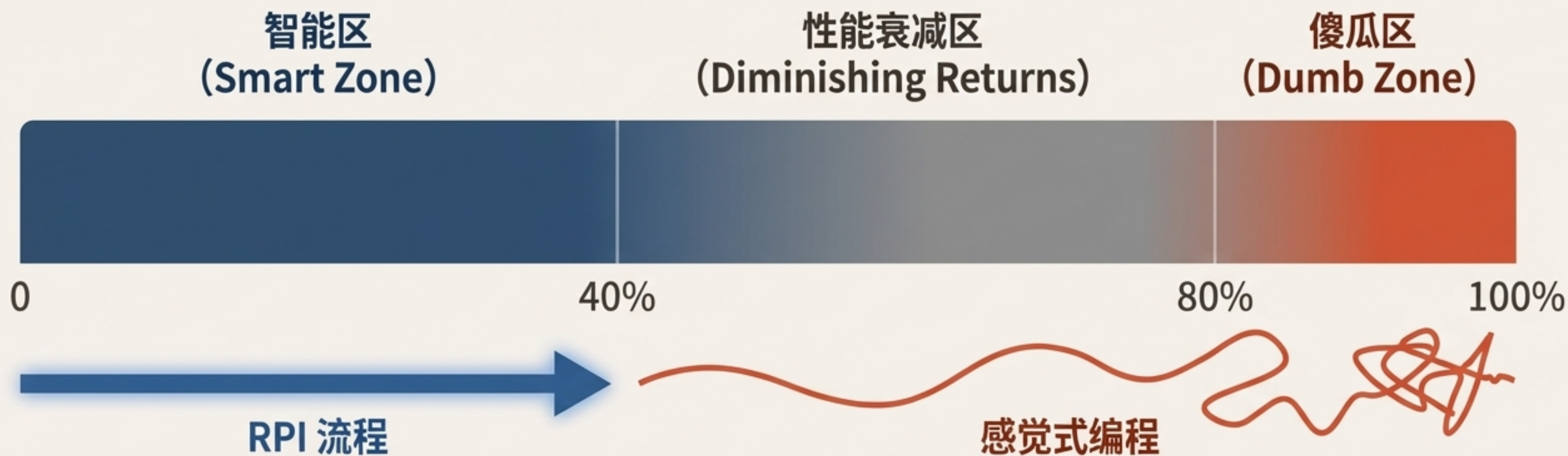
— Tobi Lütke, CEO of Shopify

一个可行的工作流：研究 → 计划 → 实现（RPI）

将与 AI 的协作分解为三个独立的、关注上下文管理的阶段，以避免进入“傻瓜区”。



原理：在“智能区”工作，规避“傻瓜区”



关键启示

“上下文压缩”是关键。RPI 流程通过在每个阶段创建简洁的文档（研究报告、计划文件），实现了对上下文的“有意压缩”，从而始终保持在智能区内工作。

构建你的上下文资产：将非结构化知识转化为工程资源

将项目中的隐性知识显性化，创建 AI 可以稳定引用的“真理来源”。



claude.md / gemini.md

全局规则

- 编码最佳实践
- 测试标准
- 风格指南
- 通用“陷阱”规避方法



任务需求文档（PRD）

产品需求

- 功能的详细描述
- 用户故事与目标
- 成功标准



project_summary.md

项目结构摘要

- AI 生成的代码库快照
- 文件树与模块职责
- 关键依赖



/examples

代码示例

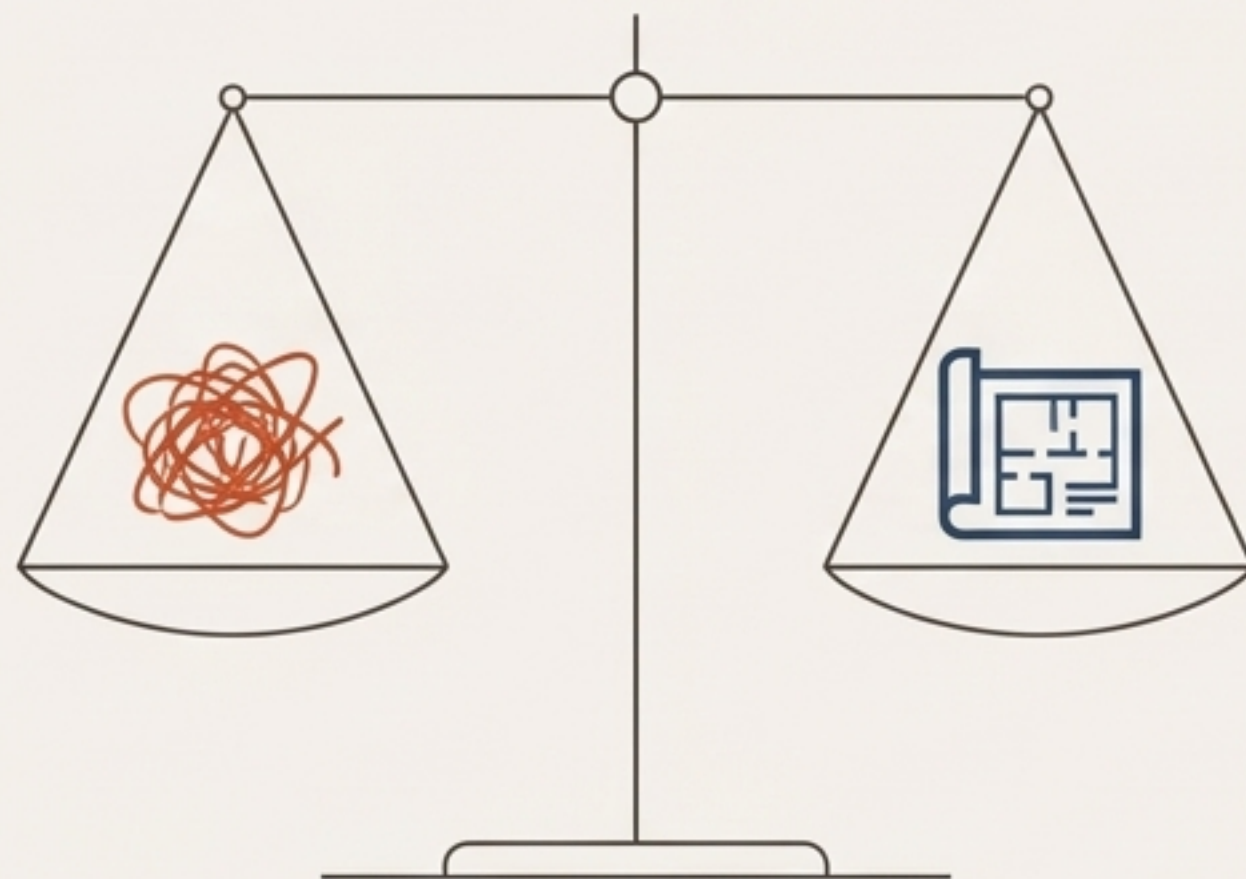
- 与当前任务相似的“黄金标准”代码片段
- 提升质量和风格一致性

这不是一场零和游戏：在正确的时间选择正确的工具

感觉式编程 (Vibe Coding)

探索
速度
原型

强大的**探索工具**。像在和一位充满创意的初级程序员头脑风暴。



上下文工程 (Context Engineering)

生产
可靠
架构

可靠的**生产工具**。像在管理一个需要明确指令和蓝图的高效施工团队。

核心技能

现代工程师的关键技能，是判断何时应该“Vibe”，何时需要“Engineering”。
这需要对任务的复杂性、代码的生命周期和团队协作要求有深刻的理解。

未来的我们：从“感觉式编程”到“感觉式工程” (Vibe Engineering)



新角色定义：指导与架构

我们的主要工作是设计系统、提供高质量的上下文、验证 AI 的输出，并做出关键的架构决策。我们像“敏锐的观察者”，监督 AI Agents 工作。



工作模式的演变

- **并行探索**：同时启动多个 AI agent 解决同一问题，然后选择最优方案。
- **异步委托**：将复杂的、长周期的任务交给 AI，自己则专注于更高层次的战略思考。

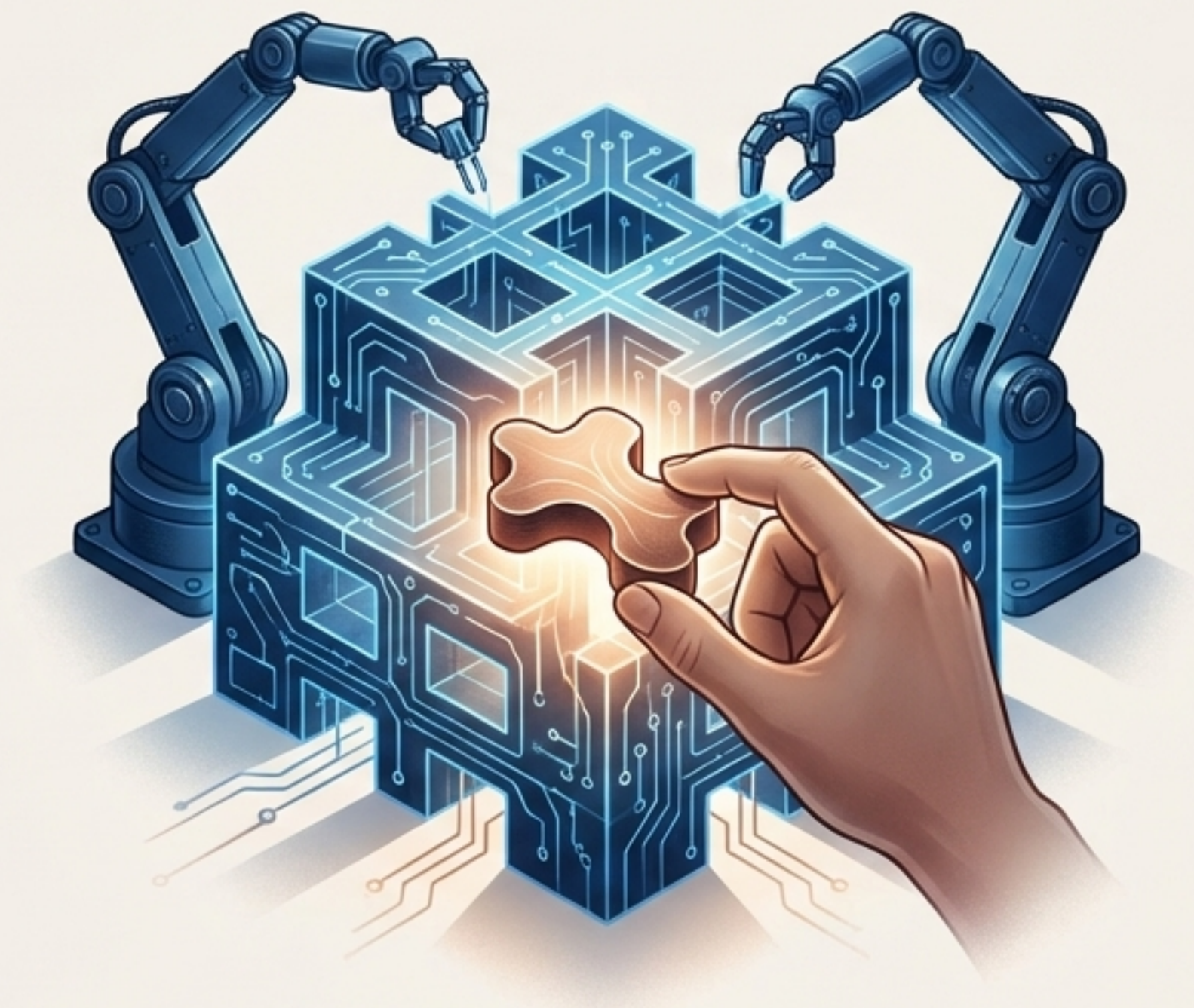
“我们工程师都在‘升职’——变成了管理者，甚至总监。因为你要用子 Agent、用任务生成任务的方式来推进工作。”

—— Aaron Friel, OpenAI

人类在环路中的终极价值：品味、判断力与对“看不见的世界”的洞察

AI 的局限

LLM 只能看到你提供给它的数据。它无法理解代码库之外的商业背景、团队动态、客户的真实需求和未来的战略方向。



工程师的独特价值

品味 (Taste)：对“好的设计”的直觉判断。

判断力 (Judgment)：在无穷的可能性中做出权衡，选择“正确”的技术路径。

上下文供给 (Context Provisioning)：识别并提供模型“看不见的世界”中的关键信息。

我们的工作不是被取代，而是被提升到一个更具战略性的层面。